
MANUAL DE USUARIO

para

Sistema de Lectura de Crotales de Animales

Version 1.0

Preparado por

Andrea Celeste Curcio, Jaume Adrover

Jaudre CV Services
Campus de Móstoles
Calle Tulipán s/n
28933 Móstoles, Madrid

27 de abril de 2026

Historial de versiones

Nombre	Fecha	Cambios	Version
Jaume Adrover	2026-04-23	Versión inicial del documento	1.0

Índice general

1	Introducción	3
2	Guía del instalador	4
2.1	Acceso a la Raspberry Pi	4
2.2	Instalación de dependencias	4
2.3	Clonación del repositorio	5
2.4	Preparación del directorio de datos	5
2.5	Construcción y ejecución con Docker	6
2.6	Ejecución alternativa sin Docker	6
2.7	Notas importantes	7
2.8	Arquitectura de despliegue	8
3	Guía del usuario	9
3.1	Ejecución sobre una imagen	9
3.2	Procesamiento por lotes (batch)	10
3.3	Notas importantes	11
3.4	Problemas comunes	11

1 Introducción

Este proyecto implementa una herramienta para la lectura automática de crotales bovinos mediante técnicas de visión artificial y reconocimiento óptico de caracteres (OCR). Su objetivo es detectar y extraer el identificador numérico presente en los crotales a partir de imágenes.

El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

https://github.com/jaaumeadrover/AIVA_2026-BovineVision

¿Qué es esta herramienta?

Se trata de una aplicación que procesa imágenes de crotales y extrae automáticamente el número identificador mediante un flujo de preprocesamiento y OCR. Puede trabajar tanto con imágenes individuales como con conjuntos de imágenes (modo batch).

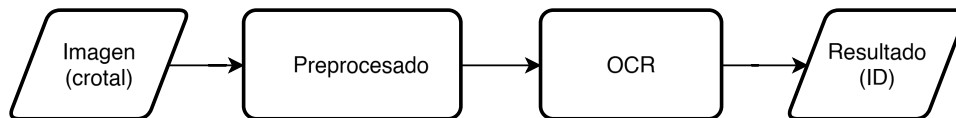


Figura 1: Flujo de procesamiento del sistema OCR

¿Para qué sirve?

Permite automatizar la lectura de identificadores de animales, evitando la intervención manual. Esto facilita tareas como:

- Procesar grandes cantidades de imágenes de forma automática
- Reducir errores en la lectura de identificadores
- Generar resultados estructurados (archivos de salida)

Utilidad para el usuario

El usuario puede ejecutar la aplicación sobre imágenes de crotales y obtener el identificador detectado. Además, es posible validar los resultados frente a un conjunto de datos de referencia (dataset) y detectar posibles duplicados en el procesamiento por lotes.

El uso de la herramienta se describe paso a paso en las secciones siguientes de este manual.

2 Guía del instalador

En esta sección se describe paso a paso cómo instalar la aplicación y ponerla en funcionamiento en la Raspberry Pi.

Nota: Aunque el sistema está diseñado para ejecutarse en una Raspberry Pi, el pipeline OCR puede probarse en un ordenador personal con Python y las dependencias instaladas. Esto permite validar el funcionamiento antes del despliegue final.

2.1. Acceso a la Raspberry Pi

Antes de comenzar, es necesario acceder a la Raspberry Pi donde se ejecutará el sistema.

Opción A: Acceso remoto (recomendado)

Desde otro ordenador, abrir una terminal y conectarse mediante SSH:

```
ssh usuario@raspberrypi.local
```

Al conectarse, se solicitará la contraseña del usuario configurada en la Raspberry Pi.

Nota: Si es la primera conexión, puede aparecer un aviso de seguridad. En ese caso, escribir *yes* y pulsar Enter.

Opción B: Acceso directo

Si la Raspberry dispone de pantalla y teclado, abrir directamente la aplicación *Terminal*.

2.2. Instalación de dependencias

Git

Comprobar si Git está instalado:

```
git --version
```

Si no está instalado:

```
sudo apt update  
sudo apt install git
```

Docker

Instalar Docker:

```
curl -fsSL https://get.docker.com | sh
```

Configurar permisos (recomendado):

```
sudo usermod -aG docker $USER
```

Tras este paso, los comandos de Docker podrán ejecutarse sin necesidad de utilizar `sudo`.
Cerrar sesión y volver a iniciarla.
Comprobar instalación:

```
docker --version
```

2.3. Clonación del repositorio

Una vez dentro de la Raspberry Pi (o desde una terminal conectada por SSH), ejecutar:

```
git clone https://github.com/jaaumeadrover/AIVA_2026-BovineVision  
cd AIVA_2026-BovineVision
```

Nota: También es posible clonar el repositorio mediante herramientas gráficas como Py-Charm, aunque no es necesario para ejecutar el sistema.

2.4. Preparación del directorio de datos

El sistema utiliza una carpeta llamada `data` para almacenar las imágenes de entrada y los resultados generados.

Si la carpeta no existe en el repositorio, debe crearse manualmente:

```
mkdir data
```

Esta carpeta se utilizará para:

- Almacenar imágenes de entrada
- Guardar los resultados generados (archivos `.txt` o `.csv`)
- Intercambiar datos entre el sistema host y el contenedor Docker

Ejemplo de uso:

- Entrada: `data/imagen.TIF`
- Salida: `data/resultado.txt`

2.5. Construcción y ejecución con Docker

Construcción de la imagen

Dentro del directorio del proyecto:

```
docker build -t myapp .
```

Nota: En algunos sistemas es necesario ejecutar los comandos de Docker con privilegios de administrador. En ese caso, se debe anteponer **sudo** al comando.

Por ejemplo:

```
sudo docker build -t myapp .
```

El comando **sudo** permite ejecutar instrucciones con permisos elevados del sistema. Este proceso puede tardar varios minutos.

Ejecución del contenedor

```
docker run -it -v $(pwd)/data:/app/data myapp
```

Este comando:

- Inicia el contenedor en modo interactivo
- Vincula la carpeta **data** con el contenedor

Una vez iniciado, se abrirá una terminal dentro del contenedor. Desde ahí se pueden ejecutar los scripts del sistema, por ejemplo:

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

2.6. Ejecución alternativa sin Docker

Si Docker no funciona o se desea realizar pruebas rápidas, el sistema puede ejecutarse directamente en el ordenador sin utilizar contenedores.

En este caso, los comandos se ejecutan desde la terminal del sistema (fuera de Docker).

1. Crear entorno virtual (opcional)

```
python -m venv venv  
source venv/bin/activate % Linux / Mac  
venv\Scripts\activate % Windows
```

2. Instalar dependencias

```
pip install -r requirements.txt
```

3. Preparar la carpeta de datos

Asegurarse de que existe la carpeta `data`. Si no existe, crearla:

```
mkdir data
```

4. Ejecutar prueba

El repositorio incluye una imagen de ejemplo en `data/0001.TIF`, que puede utilizarse para comprobar el funcionamiento del sistema.

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

Nota: El usuario puede sustituir esta imagen por cualquier otra imagen de crotal colocándola en la carpeta `data`.

Si todo funciona correctamente, se generará un archivo `resultado.txt` en la carpeta `data` con el identificador detectado.

2.7. Notas importantes

General

- La carpeta `data` es obligatoria para la entrada y salida de datos.
- Asegúrese de que las rutas de entrada y salida son correctas.
- El sistema puede ejecutarse tanto con Docker como directamente en local.
- Las imágenes de entrada deben colocarse en la carpeta `data`.

Ejecución con Docker

- El flag `-it` permite interactuar con la terminal del contenedor.
- El volumen `-v $(pwd)/data:/app/data` permite compartir archivos entre el sistema y el contenedor.
- `$(pwd)` hace referencia al directorio actual desde el que se ejecuta el comando.

2.8. Arquitectura de despliegue

El sistema se ejecuta sobre una Raspberry Pi 5 con sistema operativo Raspbian OS de 64 bits. En este dispositivo se utiliza un contenedor Docker que encapsula la aplicación y sus dependencias.

Dentro del contenedor se incluyen librerías como EasyOCR, OpenCV y NumPy, junto con el script principal `predict.py`, encargado del procesamiento de las imágenes.

El sistema utiliza una carpeta compartida (`/data`) que permite intercambiar archivos entre la Raspberry Pi y el contenedor.

El sistema no incluye un mecanismo de captura automática de imágenes. Las imágenes de entrada deben ser proporcionadas externamente, ya sea mediante copia manual o transferencia desde otro dispositivo.

En este contexto, un dispositivo externo (denominado *Data publisher*) puede encargarse de enviar imágenes a la Raspberry Pi a través de la red local (por ejemplo, mediante SFTP). Este componente no forma parte del sistema principal, pero permite automatizar la entrada de datos.

Esta arquitectura permite desacoplar el procesamiento del origen de los datos, facilitando la integración con distintos sistemas de captura y mejorando la escalabilidad y el mantenimiento del sistema.

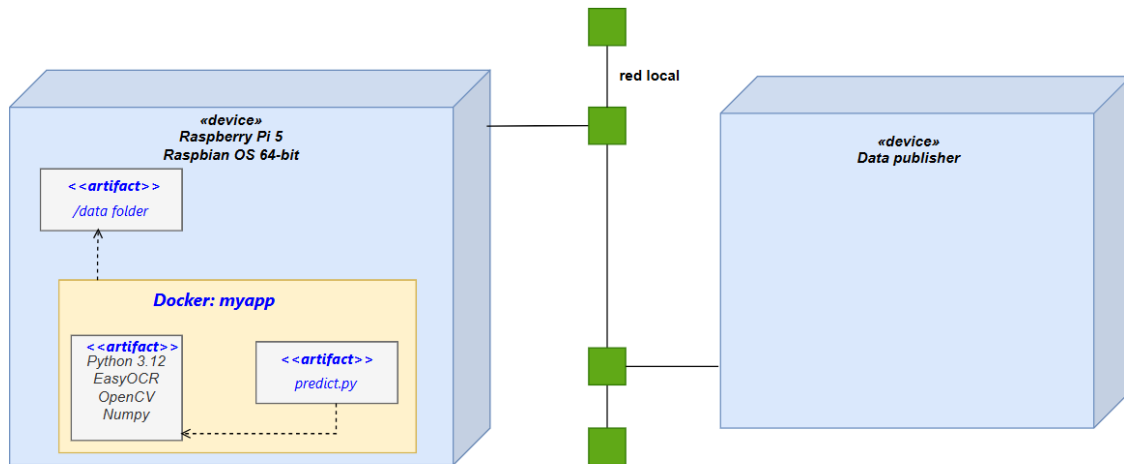


Figura 2: Arquitectura de despliegue del sistema

3 Guía del usuario

En esta sección se describe cómo utilizar el sistema una vez instalado.

Se asume que la instalación se ha realizado correctamente siguiendo la Guía del instalador.

Los comandos pueden ejecutarse:

- Dentro del contenedor Docker
- O directamente en el sistema local (ejecución con Python)

3.1. Ejecución sobre una imagen

1. Preparar la imagen de entrada

Copiar la imagen que se desea procesar en la carpeta `data` del proyecto.

Ejemplo:

```
data/imagen nueva.TIF
```



Figura 3: Imagen de ejemplo de un crotal utilizada como entrada del sistema.

2. Ejecutar el sistema OCR

Ejecutar el siguiente comando:

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

Este comando procesa la imagen de entrada y, como se muestra en la Figura 5, tras la ejecución se genera el archivo de salida en la carpeta `data`.

Nota: El archivo `0001.TIF` es un ejemplo incluido en el repositorio. Puede sustituirse por cualquier otra imagen de crotal colocada en la carpeta `data`, modificando el nombre del archivo en el comando. De igual forma, el nombre del archivo de salida puede personalizarse según se desee.

```

Progress: | 100.0% Complete --- Procesando imagen: 0001.TIF ---
C:\Users\andre\AIVA_2026-BovineVision\venv\Lib\site-packages\torch\utils\data\loader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
Resultado: 0288 (168.48s)

```

Figura 4: Salida en terminal del sistema durante la ejecución, donde se muestra el identificador detectado correctamente (0288).

 0001.TIF	30/05/2016 1:27	TIFImage.Document
 resultado.txt	26/04/2026 19:42	Documento de tex...

Figura 5: Contenido de la carpeta `data` tras la ejecución del sistema, incluyendo la imagen de entrada y el archivo de salida generado.

3. Verificar el resultado

Para visualizar el resultado:

```
cat data/resultado.txt
```

El contenido del archivo puede observarse en la Figura 6, donde se muestra el identificador detectado correctamente.

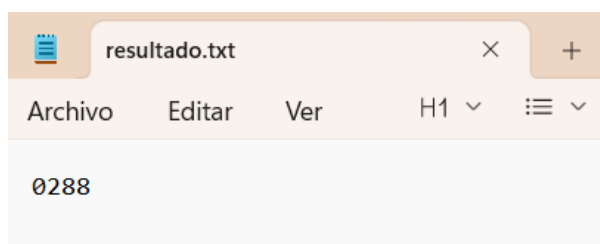


Figura 6: Contenido del archivo `resultado.txt`, donde el sistema detecta correctamente el identificador 0288.

3.2. Procesamiento por lotes (batch)

El sistema permite procesar múltiples imágenes de forma automática.

1. Preparar las imágenes

Copiar todas las imágenes a procesar dentro de la carpeta `data`.

2. Ejecutar en modo batch

```
python -m src.predict --dir data --eval data/resultados.csv
```

3. Ver resultados

```
cat data/resultados.csv
```

El archivo generado contiene:

- Nombre de la imagen
- Resultado OCR
- Validación (si aplica)
- Tiempo de procesamiento

3.3. Notas importantes

- Todas las imágenes deben estar dentro de la carpeta **data**.
- Los resultados se guardan en la misma carpeta.
- Verificar que las rutas de entrada y salida son correctas.
- Si se utiliza Docker, asegurarse de que el contenedor está en ejecución antes de lanzar los comandos.

3.4. Problemas comunes

- **Error: No se encuentra la imagen** Verificar que:
 - La carpeta **data** existe
 - La imagen está dentro de **data**
 - El nombre del archivo coincide exactamente (incluyendo mayúsculas/minúsculas)
- **Error al ejecutar Docker** Asegurarse de que Docker está en ejecución.
- **No se genera archivo de salida** Verificar que:
 - La ruta de salida es correcta
 - El script se ha ejecutado sin errores